28.4 高级记忆功能

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~380k tokens，输出 ~9k tokens（本节）

前三节覆盖了记忆系统的基础架构：Markdown 文件分块、向量嵌入、混合搜索。本节分析五个高级功能：批量嵌入 API、嵌入缓存、会话记忆、QMD 外部后端和搜索管理器——这些功能将基础架构从"能用"推向"好用"。

28.4.1 批量嵌入（Batch Indexing）

为什么需要批量 API?

对于记忆文件很多的用户，重建索引可能产生数百甚至数千个文本块。如果逐一调用嵌入 API，每个块一次 HTTP 请求，不仅速度慢，还容易触发速率限制。云端提供商（OpenAI、Gemini、Voyage）都提供了**异步批量 API（Batch API）**——一次提交大量请求，服务端在后台处理，完成后一次性返回所有结果。

统一的批量流程

三家提供商的批量 API 虽然细节不同，但 OpenClaw 将它们封装为相同的流程模式：

```
1. 准备请求 → 构建 JSONL 文件（每行一个嵌入请求）
2. 上传文件 → 通过 Files API 上传 JSONL
3. 创建批量任务 → 调用 Batches API 提交任务
4. 轮询等待 → 定期检查任务状态直到完成
5. 下载结果 → 获取输出文件，解析每行的嵌入向量
6. 结果映射 → 通过 custom_id 将向量与原始文本块关联
```

以 OpenAI 为例：

```typescript
// src/memory/batch-openai.ts（简化）

export type OpenAiBatchRequest = {
  custom_id: string;          // 唯一标识，用于结果关联
  method: "POST";
  url: "/v1/embeddings";
  body: { model: string; input: string };
};

export async function runOpenAiEmbeddingBatches(params: {
  openAi: OpenAiEmbeddingClient;
  requests: OpenAiBatchRequest[];
  wait: boolean;              // 是否等待完成
  pollIntervalMs: number;     // 轮询间隔（默认 2s）
  timeoutMs: number;          // 超时时间（默认 60min）
  concurrency: number;        // 并行批次数
}): Promise<Map<string, number[]>> {
  // 1. 按 50,000 条拆分为多组（OpenAI 单批上限）
  const groups = splitOpenAiBatchRequests(params.requests);
  const byCustomId = new Map<string, number[]>();

  const tasks = groups.map(group => async () => {
    // 2. 上传 JSONL → 3. 创建批量任务
    const batchInfo = await submitOpenAiBatch({ openAi, requests: group });

    // 4. 轮询等待完成
    const completed = await waitForOpenAiBatch({
      batchId: batchInfo.id,
      pollIntervalMs, timeoutMs,
    });

    // 5. 下载并解析结果
    const content = await fetchOpenAiFileContent({ fileId: completed.outputFileId });
    const outputLines = parseOpenAiBatchOutput(content);

    // 6. 映射结果
    for (const line of outputLines) {
      const embedding = line.response?.body?.data?.[0]?.embedding ?? [];
      byCustomId.set(line.custom_id, embedding);
    }
  });

  // 多组并行执行
```

```
  await runWithConcurrency(tasks, params.concurrency);
  return byCustomId;
}
```

提供商差异

特性	OpenAI	Gemini	Voyage
单批上限	50,000	50,000	50,000
完成窗口	24 小时	无限制	12 小时
API 密钥头	`Authorization: Bearer`	`x-goog-api-key`	`Authorization: Bearer`
上传格式	FormData + JSONL	multipart/related	FormData + JSONL
状态字段	`status` (completed)	`state` (SUCCEEDED)	`status` (completed)
结果 ID	`custom_id`	`key` / `custom_id`	`custom_id`
批量端点	`/v1/batches`	`asyncBatchEmbedContent`	`/v1/batches`

Gemini 的实现最为独特——它使用 `multipart/related` 格式上传（而非 FormData），批量端点挂在模型路径下（`models/gemini-embedding-001:asyncBatchEmbedContent`），且状态值使用大写（`SUCCEEDED` / `FAILED`）。

批量失败容错

批量 API 可能因各种原因失败（网络超时、API 不可用、配额耗尽）。OpenClaw 实现了**渐进式禁用**机制：

```
// src/memory/manager.ts（简化）

const BATCH_FAILURE_LIMIT = 2;

private async recordBatchFailure(params: {
  provider: string;
  message: string;
  forceDisable?: boolean;
}): Promise<{ disabled: boolean; count: number }> {
  this.batchFailureCount += 1;
  const disabled = params.forceDisable
    || this.batchFailureCount >= BATCH_FAILURE_LIMIT;
  if (disabled) {
    this.batch.enabled = false;  // 禁用批量，回退到逐条嵌入
  }
  return { disabled, count: this.batchFailureCount };
}
```

规则很简单：**累计 2 次失败后自动禁用批量 API**，回退到逐条 `embedBatch` 调用。特殊情况如 Gemini 的 `asyncBatchEmbedContent not available` 错误（模型不支持批量）会**立即禁用**。成功的批量操作会重置失败计数。

整个批量执行还包裹在 `runBatchWithFallback` 中——即使批量 API 完全不可用，系统也能透明地回退到逐条嵌入：

```
// src/memory/manager.ts（简化）

private async runBatchWithFallback(params) {
  if (!this.batch.enabled) {
    return await params.fallback();        // 直接用逐条嵌入
  }
  try {
    const result = await this.runBatchWithTimeoutRetry(params);
    await this.resetBatchFailureCount();   // 成功，重置计数
    return result;
  } catch (err) {
    await this.recordBatchFailure({ ... });
    return await params.fallback();        // 失败，回退到逐条
  }
}
```

28.4.2 嵌入缓存机制

缓存的必要性

每次同步记忆文件时，未变更的文本块不需要重新计算嵌入。 `embedding_cache` 表基于**文本哈希**缓存已计算的向量：

```
// embedding_cache 表的复合主键
PRIMARY KEY (provider, model, provider_key, hash)
```

四个字段共同确定一个缓存条目：

- `provider` ： 嵌入提供者（openai/gemini/voyage/local）
- `model` ： 具体模型名（text-embedding-3-small 等）
- `provider_key` ： 提供者配置的指纹（如 API baseUrl 的哈希），确保不同配置的同名模型不会混用
- `hash` ： 文本内容的 SHA-256 哈希

缓存命中流程

索引文件时， `embedChunksInBatches` 先查缓存再调 API：

```
// src/memory/manager.ts（简化）

private async embedChunksInBatches(chunks: MemoryChunk[]): Promise<number[][]> {
  // 1. 从缓存加载已有的嵌入
  const cached = this.loadEmbeddingCache(chunks.map(c => c.hash));
  const embeddings: number[][] = Array(chunks.length).fill([]);
  const missing: Array<{ index: number; chunk: MemoryChunk }> = [];

  // 2. 缓存命中的直接使用，未命中的收集起来
  for (let i = 0; i < chunks.length; i++) {
    const hit = cached.get(chunks[i].hash);
    if (hit && hit.length > 0) {
      embeddings[i] = hit;           // 缓存命中
    } else {
      missing.push({ index: i, chunk: chunks[i] }); // 缓存未命中
    }
  }

  if (missing.length === 0) return embeddings; // 全部命中，无需 API 调用

  // 3. 只对未命中的块调用嵌入 API
  const batches = this.buildEmbeddingBatches(missing.map(m => m.chunk));
  const toCache = [];
  for (const batch of batches) {
    const batchEmbeddings = await this.embedBatchWithRetry(batch.map(c => c.text));
    // 将结果填回正确位置，同时收集要缓存的条目
    for (let i = 0; i < batch.length; i++) {
      embeddings[missing[cursor + i].index] = batchEmbeddings[i];
      toCache.push({ hash: batch[i].hash, embedding: batchEmbeddings[i] });
    }
  }

  // 4. 将新计算的嵌入写入缓存
  this.upsertEmbeddingCache(toCache);
  return embeddings;
}
```

缓存清理

缓存表不会无限增长。 `pruneEmbeddingCacheIfNeeded` 使用 LRU 策略清理过期条目：

```
// src/memory/manager.ts（简化）

private pruneEmbeddingCacheIfNeeded(): void {
  const count = this.db.prepare(
    `SELECT COUNT(*) as c FROM embedding_cache`
  ).get().c;

  if (count <= this.cache.maxEntries) return;

  const excess = count - this.cache.maxEntries;
  this.db.prepare(
    `DELETE FROM embedding_cache
     WHERE rowid IN (
       SELECT rowid FROM embedding_cache
       ORDER BY updated_at ASC      -- 最久未使用的先删除
       LIMIT ?
     )`
  ).run(excess);
}
```

> **衍生解释**：LRU（Least Recently Used）是一种缓存淘汰策略——当缓存满时，优先删除最久未被访问的条目。这里通过 `updated_at` 字段实现：每次缓存命中或写入都会更新时间戳，清理时按时间戳升序删除最旧的条目。

缓存在批量查询中的优化

`loadEmbeddingCache` 分批查询（每批 400 条），避免 SQL `IN` 子句过长：

```
const batchSize = 400;
for (let start = 0; start < unique.length; start += batchSize) {
  const batch = unique.slice(start, start + batchSize);
  const placeholders = batch.map(() => "?").join(", ");
  const rows = this.db.prepare(
    `SELECT hash, embedding FROM embedding_cache
     WHERE provider = ? AND model = ? AND provider_key = ?
       AND hash IN (${placeholders})`
  ).all(...baseParams, ...batch);
}
```

28.4.3 会话记忆搜索（实验性）

为什么索引会话记录?

记忆文件记录的是用户主动保存的信息。但大量有价值的上下文存在于**历史对话**中——用户曾经讨论的技术方案、解决过的 bug、做出的决策。会话记忆搜索允许 Agent 检索这些历史对话片段。

会话文件格式

OpenClaw 的会话记录存储为 JSONL 文件（每行一个 JSON 对象），位于 Agent 的 transcripts 目录：

```
~/.openclaw/state/agents/{agentId}/transcripts/
├── session-abc123.jsonl
├── session-def456.jsonl
└── ...
```

`buildSessionEntry` 解析 JSONL 文件，提取 `user` 和 `assistant` 角色的文本内容，并进行敏感信息脱敏：

```
// src/memory/session-files.ts（简化）

export async function buildSessionEntry(absPath: string): Promise<SessionFileEntry | nu
  const raw = await fs.readFile(absPath, "utf-8");
  const lines = raw.split("\n");
  const collected: string[] = [];

  for (const line of lines) {
    const record = JSON.parse(line);
    if (record.type !== "message") continue;

    const { role, content } = record.message;
    if (role !== "user" && role !== "assistant") continue;

    const text = extractSessionText(content);
    if (!text) continue;

    const safe = redactSensitiveText(text, { mode: "tools" }); // 脱敏
    const label = role === "user" ? "User" : "Assistant";
    collected.push(`${label}: ${safe}`);
  }

  return {
    path: `sessions/${path.basename(absPath)}`,
    absPath,
    hash: hashText(collected.join("\n")),
    content: collected.join("\n"),
  };
}
```

注意 `redactSensitiveText` 调用——会话记录可能包含 API Key、密码等敏感信息，脱敏后再写入索引。

增量同步

会话文件的同步采用 delta 机制：只有当文件大小或消息数超过阈值时才重新索引，避免频繁的对话更新触发全量重建。会话 chunks 使用 `source: "sessions"` 标记，与记忆 chunks（`source: "memory"`）在同一个 SQLite 数据库中共存但可以独立过滤。

28.4.4 QMD 后端（BM25 + 向量 + 重排序）

什么是 QMD?

QMD 是一个**外部 CLI 工具**，提供比内置引擎更强大的搜索能力——BM25 全文搜索 + 向量搜索 + 重排序（Reranking）。OpenClaw 通过 `QmdMemoryManager` 类集成它，作为内置记忆引擎的高级替代。

> **衍生解释**：重排序（Reranking）是信息检索中的一种两阶段策略。第一阶段（召回）用 BM25 或向量搜索快速获取大量候选；第二阶段（重排序）用更精确但更慢的模型（如交叉编码器 Cross-Encoder）对候选重新排名。这类似于搜索引擎先用倒排索引粗筛，再用机器学习模型精排。

架构设计

`QmdMemoryManager` 通过 `child_process.spawn` 调用外部 `qmd` 命令，实现了与内置引擎相同的 `MemorySearchManager` 接口：

```typescript
// src/memory/qmd-manager.ts（简化）

export class QmdMemoryManager implements MemorySearchManager {
  private readonly env: NodeJS.ProcessEnv;  // XDG 环境变量

  async search(query: string, opts?) {
    const args = ["query", query, "--json", "-n", String(limit)];
    const { stdout } = await this.runQmd(args, { timeoutMs });
    const parsed = JSON.parse(stdout) as QmdQueryResult[];
    // 解析结果、路径映射、分数过滤...
    return results;
  }

  async sync(params?) {
    await this.runQmd(["update"], { timeoutMs: 120_000 });
    // 按配置间隔执行 embed 命令
    if (shouldEmbed) {
      await this.runQmd(["embed"], { timeoutMs: 120_000 });
    }
  }

  private async runQmd(args: string[], opts?) {
    return new Promise((resolve, reject) => {
      const child = spawn(this.qmd.command, args, {
        env: this.env,      // XDG_CONFIG_HOME, XDG_CACHE_HOME
        cwd: this.workspaceDir,
      });
      // 收集 stdout/stderr，超时 SIGKILL...
    });
  }
}
```

集合（Collection）管理

QMD 使用**集合**概念组织索引源。每个集合对应一个目录和文件匹配模式：

```
// 默认集合布局
collections = [
  { name: "memory-root", path: workspaceDir, pattern: "MEMORY.md" },
  { name: "memory-alt",  path: workspaceDir, pattern: "memory.md" },
  { name: "memory-dir",  path: workspaceDir + "/memory", pattern: "**/*.md" },
  // 如果启用会话索引:
  { name: "sessions",    path: sessionsExportDir, pattern: "**/*.md" },
];
```

集合通过 `qmd collection add` 命令创建，OpenClaw 在初始化时检查并确保所有配置的集合存在。

会话导出

QMD 不直接读取 JSONL 会话文件——它需要 Markdown。 `exportSessions` 方法将会话记录转换为 Markdown 文件，存放到导出目录供 QMD 索引：

```
private async exportSessions(): Promise<void> {
  const files = await listSessionFilesForAgent(this.agentId);
  for (const sessionFile of files) {
    const entry = await buildSessionEntry(sessionFile);
    if (cutoff && entry.mtimeMs < cutoff) continue;     // 过期跳过
    const target = path.join(exportDir, `${basename}.md`);
    await fs.writeFile(target, this.renderSessionMarkdown(entry)); // JSONL → Markdown
  }
  // 清理导出目录中不再需要的旧文件
}
```

定时更新

QMD 的索引更新通过两个命令完成： `qmd update` （扫描文件变更）和 `qmd embed` （生成嵌入向量）。更新策略：

参数	默认值	说明
`update.onBoot`	`true`	启动时立即执行 update
`update.intervalMs`	5 分钟	定期执行 update
`update.debounceMs`	15 秒	防抖：距上次更新不足此间隔则跳过
`update.embedIntervalMs`	60 分钟	embed 命令的最小执行间隔

embed 间隔远大于 update 间隔——因为文件扫描很快，但嵌入计算较慢。

作用域控制

QMD 支持基于会话键的**作用域控制**，限制哪些聊天上下文可以使用 QMD 搜索：

```
// 默认作用域：只允许直接对话，禁止群组
const DEFAULT_QMD_SCOPE = {
  default: "deny",
  rules: [
    { action: "allow", match: { chatType: "direct" } },
  ],
};
```

28.4.5 搜索管理器（search-manager.ts）

两级管理架构

`getMemorySearchManager` 是记忆搜索的最终入口。它决定使用 QMD 还是内置引擎，并在两者之间提供透明的故障转移：

```typescript
// src/memory/search-manager.ts（简化）

export async function getMemorySearchManager(params: {
  cfg: OpenClawConfig;
  agentId: string;
}): Promise<MemorySearchManagerResult> {
  const resolved = resolveMemoryBackendConfig(params);

  if (resolved.backend === "qmd" && resolved.qmd) {
    // 优先使用 QMD
    const primary = await QmdMemoryManager.create({ cfg, agentId, resolved });
    if (primary) {
      // 用 FallbackMemoryManager 包装
      return { manager: new FallbackMemoryManager({
        primary,
        fallbackFactory: async () => {
          const { MemoryIndexManager } = await import("./manager.js");
          return await MemoryIndexManager.get(params);
        },
      }) };
    }
  }

  // QMD 不可用或未配置，使用内置引擎
  const { MemoryIndexManager } = await import("./manager.js");
  return { manager: await MemoryIndexManager.get(params) };
}
```

FallbackMemoryManager

`FallbackMemoryManager` 实现了**透明故障转移（Transparent Failover）**模式：

```typescript
// src/memory/search-manager.ts（简化）

class FallbackMemoryManager implements MemorySearchManager {
  private primaryFailed = false;
  private fallback: MemorySearchManager | null = null;

  async search(query, opts?) {
    if (!this.primaryFailed) {
      try {
        return await this.deps.primary.search(query, opts);
      } catch (err) {
        this.primaryFailed = true;         // 标记主引擎失败
        this.lastError = err.message;
        await this.deps.primary.close();   // 关闭主引擎
      }
    }
    // 懒创建回退引擎
    const fallback = await this.ensureFallback();
    return await fallback.search(query, opts);
  }

  private async ensureFallback(): Promise<MemorySearchManager | null> {
    if (!this.fallback) {
      this.fallback = await this.deps.fallbackFactory(); // 动态导入内置引擎
    }
    return this.fallback;
  }
}
```

这个设计有三个精妙之处：

1. **懒回退**：只有主引擎（QMD）实际失败后才创建回退引擎（内置），避免浪费资源。
2. **永久切换**：一旦 `primaryFailed` 为 `true`，后续所有搜索都走回退路径，不会反复尝试已失败的主引擎。
3. **接口统一**：对调用者完全透明——无论底层是 QMD、内置还是回退后的内置，`search()` 签名和返回类型完全相同。

缓存策略

搜索管理器使用 `QMD_MANAGER_CACHE` 缓存已创建的管理器实例：

```
const QMD_MANAGER_CACHE = new Map<string, MemorySearchManager>();
```

缓存键由 `agentId` 和 QMD 配置的稳定序列化（排序后 JSON）组成，确保：

- 同一 Agent 同一配置复用同一实例
- 配置变更自动创建新实例
- `close()` 时从缓存中移除

本节小结

1. **批量嵌入 API** 将数百个嵌入请求打包为一次异步任务，支持 OpenAI、Gemini、Voyage 三家提供商。失败累计 2 次后自动禁用，透明回退到逐条嵌入。
2. **嵌入缓存**基于文本哈希的四维复合键（provider + model + provider_key + hash），索引时先查缓存再调 API，LRU 策略控制缓存大小。
3. **会话记忆搜索**将 JSONL 对话记录解析为文本块并索引，支持敏感信息脱敏和增量同步，使 Agent 能够检索历史对话内容。
4. **QMD 后端**通过外部 CLI 提供 BM25 + 向量 + 重排序的三阶段搜索，使用集合管理多源数据，定时更新索引，支持作用域控制。
5. **搜索管理器**实现两级架构：QMD 优先 → 内置回退。 `FallbackMemoryManager` 提供透明故障转移，确保搜索功能永远可用。
6. **降级哲学**贯穿整个记忆系统——从嵌入提供者、向量搜索、批量 API 到后端选择，每个层级都有回退方案，体现了**可用性优先**的设计原则。